Entangled Enhancement in Modular Autonomous Driving Systems: An Empirical Study of Cascade Effects from Single-Component Retraining

Rashedul Arefin Ifty, Fumio Machida
University of Tsukuba
Tsukuba, Japan
ifty.rashedul@sd.cs.tsukuba.ac.jp, machida@cs.tsukuba.ac.jp

***Abstract*—Machine learning systems in autonomous driving are maintained incrementally: one component is retrained as its domain drifts while the rest stays fixed. This pattern is incompatible with monolithic end-to-end (E2E) models, in which the smallest retrainable unit is the entire network. We examine the effect of retraining a single component of a modular perception–prediction–planning pipeline under domain shift. The pipeline is C1, a YOLOv11n detector, C2, a constant-velocity predictor, and C3, a rule-based Intelligent Driver Model planner. We train C1 on Boston nuScenes scenes, fine-tune it on Singapore scenes, and measure every component in two modes, *isolated* with ground-truth inputs and *pipeline* with live upstream inputs, before and after retraining. We observe two effects related to the analytically modeled notion of *entangled enhancement*: a non-monotone cascade and a decoupling between a component's aggregate metric and its interface behavior. Retraining C1 improved C2's pipeline-mode error by 73.98% (minADE $15.19 \rightarrow 3.95$ m), yet planning error still worsened by 4.45% (L2@3s $6.30 \rightarrow 6.58$ m), while C1's own detection metric dropped by 45.66% (mAP@50 $0.119 \rightarrow 0.064$). Since C1's aggregate metric declined, the run does not meet the strict entangled-enhancement condition but exhibits the related decoupling and non-monotonicity. All isolated-mode metrics stayed bit-identical, confirming that every change traveled through the component interfaces. From five findings we explain why aggregate component metrics and pipeline behavior diverge, and propose a cascade-aware retraining assessment protocol that turns our dual-mode measurement into an admission decision for component updates.**

***Index Terms*—machine learning systems, autonomous driving, retraining, cascade effects, software reliability, modular pipelines, regression testing**

I. Introduction

Most recent autonomous driving research follows the end-to-end (E2E) paradigm, where a single jointly trained network maps raw sensor input directly to a driving plan. One survey counts more than 270 E2E driving papers [1], built around flagship architectures such as UniAD [2] and VAD [3], and on benchmark leaderboards E2E appears to lead.

Deployed systems, however, are maintained under domain drift, one component at a time: a perception model is retrained for a new city, a predictor is updated as agent statistics shift, a planner is revised for new regulations. Maintenance is thus scoped to components, and the E2E paradigm is ill-suited as an object of maintenance research for three reasons. First, an E2E model admits a single retraining action, the entire network, with no component states to threshold and no selective update; recent repair systems reflect this, fine-tuning the whole planner on synthetic failure data [4] or attaching adapters to avoid catastrophic forgetting [5]. Second, cost: a single UniAD training run requires about 144 GPU-hours [6], so a study of *repeated* retraining, the essence of maintenance, is far more expensive than in a modular pipeline. Third, the cascade effects that make maintenance hazardous do not disappear inside a monolith but become unobservable; the analysis of hidden technical debt established that "changing anything changes everything" and that correction cascades arise whenever models consume other models' outputs [7], which an E2E network conceals behind a single loss.

We therefore adopt a modular pipeline as a requirement of the method: retraining a single component is only well-posed as a research question in an architecture with separable, independently retrainable components. This work builds directly on Wang and Machida [8], who model a two-component MLS as a continuous-time Markov chain and name the failure mode we study, *entangled enhancement*: improving an upstream component on its own metric can still degrade the system, because downstream components have learned to operate on the previous upstream output. A follow-on study casts this as an availability–continuity trade-off [9]. Both are stochastic models needing empirical calibration, and no one has yet measured how a single-component retraining event propagates through a real perception–prediction–planning pipeline. That is the gap this paper fills.

This paper supplies that measurement. We build a three-component pipeline on the nuScenes dataset [10]: C1, a YOLOv11n [11] camera detector, C2, a constant-velocity trajectory predictor, and C3, a rule-based planner from the Intelligent Driver Model (IDM) family [12] whose proposal-and-score design follows PDM-Closed [13]. We train C1 on Boston scenes, fine-tune it on Singapore scenes, a well-known domain gap within nuScenes, and keep C2 and C3 frozen. The focus is on C1, the retrained unit, and C2, its direct consumer, whose input interface carries the cascade; C3 gives the system-level reading. The main instrument is dual-mode evaluation: each downstream component is measured in *isolated* mode with ground-truth inputs and *pipeline* mode with live upstream outputs. The gap between the modes

isolates the cascade contribution, and its change across a retraining event measures the cascade the retraining injects.

Our measurements yield five findings, detailed in Section V:

- **F1. (Architectural.)** The component-level retraining question is unobservable in an E2E architecture and unaffordable at E2E training costs; a modular pipeline is the enabling architecture.
- **F2. (Isolation.)** The isolation property that stochastic MLS models assume holds exactly: the isolated-mode metrics of the frozen C2 and C3 stayed bit-identical across the retraining event.
- **F3. (Cascade existence.)** Retraining C1 alone produced large, measurable changes downstream, concentrated at the first C1→C2 interface.
- **F4. (Entangled enhancement.)** The cascade is not monotone: the same event improved C2's pipeline error by 73.98% yet worsened C3's planning error by 4.45%.
- **F5. (Metric decoupling.)** C1's aggregate mAP@50 *dropped* by 45.66% even as its outputs became more useful to C2, so a component's own metric is not a reliable admission criterion.

Contributions. This paper contributes (1) a dual-mode measurement method for retraining-induced cascades in modular MLS, with an isolation check; (2) a controlled measurement, to our knowledge the first on a real perception–prediction–planning pipeline, of a single-component retraining event under a geographic domain shift, revealing a non-monotone cascade and a decoupling between aggregate and interface metrics, effects adjacent to but distinct from the strict entangled-enhancement condition; and (3) a cascade-aware retraining assessment protocol (Section VII) with a coupling factor ρ intended to calibrate the base paper's stochastic maintenance models [8], [9].

The remainder of the paper is organized as follows. Section II reviews related work. Section III formalizes the system model and the dual-mode methodology. Section IV details the experimental setup. Section V presents results, and Section VI discusses the findings and threats to validity. Section VII presents the proposed assessment protocol, and Section VIII concludes.

II. Related Work

A. End-to-End Driving and Its Maintenance Limits

UniAD put detection, tracking, mapping, motion forecasting, occupancy prediction, and planning into a single planning-oriented network [2]. VAD replaced dense rasterized scene representations with vectorized ones for efficiency [3]. SparseDrive introduced fully sparse scene representations and reported the resource profile of the flagship systems: $7.2\times$ faster training and $5\times$ faster inference than UniAD, whose training takes about 144 GPU-hours [6]. The main survey of the field lists interpretability, causal confusion, and robustness under distribution shift as open challenges, and it does not treat component-level maintenance or retraining policy at all [1].

A recent E2E repair literature addresses maintenance directly. CorrectAD builds a self-correction loop that diagnoses failures, generates targeted training video with a diffusion model, and fine-tunes the entire E2E planner on old plus new data, correcting 62.5% of failure cases on nuScenes [4]. R2SE avoids full-model retraining by freezing the generalist model and training reinforcement-learned low-rank adapter specialists on failure regions, motivated by the catastrophic forgetting that naive fine-tuning causes [5]. Both reintroduce modular structure to make maintenance tractable, motivating the modular architecture adopted in this work.

B. Evaluation of Driving Systems

Our planning metric is open-loop displacement error on nuScenes, and the limits of this protocol are well documented. Codevilla et al. showed as early as 2018 that offline prediction error correlates only weakly with actual closed-loop driving quality [14]. Zhai et al. found that a trivial MLP using only ego state, with no perception at all, matches perception-based E2E methods on nuScenes open-loop L2 [15]. Dauner et al. showed that open-loop ego-forecasting and closed-loop driving are misaligned tasks, and that a largely rule-based planner, PDM-Closed, outperformed learned planners [13]. Their later NAVSIM benchmark provides short-horizon non-reactive metrics that align far better with closed-loop performance [16]. These critiques target open-loop metrics used to *rank* architectures; our use is differential (the same frozen planner, same scenes, one upstream change), so much of that weakness cancels, and we report collision rate alongside L2 and revisit the caveat in Sections VI and VI-C.

C. Model Updates, Regression, and Interface Compatibility

What we measure at system scale has per-sample analogues in the model-update literature. Yan et al. showed that model updates cause negative flips, samples the old model handled correctly that the new model gets wrong, at rates of 4–5% even between equal-accuracy retrains of identical architectures, and they proposed positive-congruent training with focal distillation to suppress them [17]. The backward-compatible training of Shen et al. constrains a new upstream representation model so that unchanged downstream consumers keep working [18], which is the same situation as updating C1 beneath a frozen C2 and C3. On the monitoring side, the data validation system of Breck et al. runs two-sample distribution tests between pipeline stages in production [19]. On a nuScenes detector–tracker–predictor stack nearly the same shape as ours, Ivanovic et al. showed that propagating state uncertainty across the perception–prediction interface, instead of passing point estimates, improves forecasting by a wide margin [20]. These works provide mitigation techniques; this paper contributes the complementary controlled measurement of the cascade that such techniques are intended to mitigate.

D. Reliability Modeling of Machine Learning Systems

The technical-debt analysis of Sculley et al. introduced CACE, unstable data dependencies, and correction cascades

as qualitative failure modes of ML systems [7]. The base paper for this work, Wang and Machida [8], formalized imperfect retraining of a two-component MLS as a continuous-time Markov chain, named entangled enhancement, and compared progressive and conservative maintenance policies; their follow-on work casts the same problem as an availability–continuity trade-off [9]. Both are analytical models whose state space grows as 2^N for N components, and both are calibrated on synthetic transition rates. This paper provides an empirical counterpart: the isolation property we verify (F2) is the property their state factorization assumes, and the coupling factor we propose (Section VII) is a measured statistic that can replace those synthetic rates.

III. System Model and Methodology

A. Formal Pipeline Model

We model the driving stack as a composition of three functions,

$$S = C_3 \circ C_2 \circ C_1, \qquad S(x) = C_3\big(C_2(C_1(x))\big), \tag{1}$$

where x is a camera keyframe and the three components carry out the canonical perception–prediction–planning decomposition:

- **C1 (Perception):** $z_1 = C_1(x)$ is a set of detected objects, each with a class, a position, and an extent in the ego frame.
- **C2 (Prediction):** $z_2 = C_2(z_1)$ is, for every detected object, a forecast trajectory over the next several seconds.
- **C3 (Planning):** $z_3 = C_3(z_2)$ is the ego vehicle's planned trajectory, a sequence of positions over a 3 s horizon.

The intermediate signals z_1 and z_2 are the pipeline interfaces, and each is a cascade channel: a change in C_1's output distribution shifts z_1, which its direct consumer C_2 then propagates into z_2 and C_3. Fig. 1 shows the pipeline annotated with the before/after numbers measured in this study. Only C_1 is retrained (Boston $\rightarrow$ Singapore); the isolated (*iso*) metrics of the frozen C_2 and C_3 stay constant, while their pipeline (*pipe*) metrics move because their inputs shift.

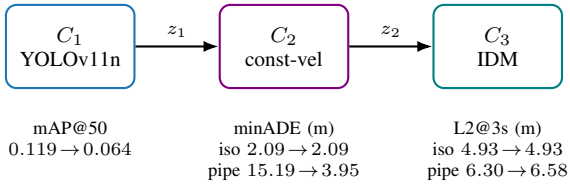


Fig. 1: Pipeline $C_1 \rightarrow C_2 \rightarrow C_3$ with before/after numbers.

B. Dual-Mode Evaluation

The core instrument is the measurement of each downstream component in two modes. Let $z_k^\star$ denote the ground-truth value of interface k. In *isolated* mode a component receives ground-truth inputs, so for C_2 and C_3 we evaluate

$$z_2^{\text{iso}} = C_2(z_1^\star), \qquad z_3^{\text{iso}} = C_3(z_2^\star). \tag{2}$$

In *pipeline* mode a component receives the live output of its upstream neighbor,

$$z_2^{\text{pipe}} = C_2(C_1(x)), \qquad z_3^{\text{pipe}} = C_3(C_2(C_1(x))). \tag{3}$$

For a component C_k with error metric M_k (lower is better), we define the *cascade gap* at C_k as

$$\Gamma_k = M_k^{\text{pipeline}} - M_k^{\text{isolated}}, \tag{4}$$

the part of C_k's error that comes from upstream imperfection rather than from C_k itself. A retraining event R applied to an upstream component changes the pipeline metrics. If the system is properly modular, it leaves all isolated metrics of untouched components unchanged. We call this the *isolation property*:

$$M_k^{\text{isolated}}(\text{before } R) = M_k^{\text{isolated}}(\text{after } R) \quad \forall \text{ frozen } C_k. \tag{5}$$

Equation (5) is not assumed but treated as a testable property of the apparatus: any leakage of the retraining event into a frozen component's isolated metric indicates contamination, and we verify the equation bit-exactly in Section V. The cascade *injected* by R at component C_k is then

$$\Delta_k = M_k^{\text{pipeline}}(\text{after } R) - M_k^{\text{pipeline}}(\text{before } R), \tag{6}$$

which, given (5), equals the change in the cascade gap Γ_k.

C. Cascade Diagnostic and Coupling Factor

The canonical cascade hypothesis is a conjunction of three conditions: the retrained upstream component improves on its own metric, the frozen terminal component is stable in isolation, and it worsens in the pipeline. Writing δ_1 for the change in the C_1 metric (positive denoting improvement) and using Δ_3 from (6), we encode the diagnostic as

$$\textsc{cascade} \equiv \big(\delta_1 > 0\big) \ \wedge \ \big(\Delta_3^{\text{iso}} = 0\big) \ \wedge \ \big(\Delta_3 > \epsilon\big), \tag{7}$$

with ϵ a small noise floor. When (7) holds, the coupling from the retrained component to the system output is summarized by the *cascade coupling factor*

$$\rho_{1\rightarrow 3} = \frac{\Delta_3}{\delta_1}. \tag{8}$$

On a synthetic pipeline with a known planted cascade, (7)–(8) recover $\rho_{1\rightarrow 3} \approx 0.84$, which serves as an apparatus check prior to the empirical run. As Section V shows, the empirical outcome does not satisfy the conjunction (7) in its canonical form: $\delta_1 < 0$ (aggregate mAP fell) while $\Delta_3 > 0$, so $\rho_{1\rightarrow 3}$ is of ambiguous sign. This mismatch is not a null result but the entangled-enhancement regime itself, and it is precisely why ρ must be interpreted together with the full cascade vector rather than as an isolated scalar (Section VII).

D. Single-Component Retraining Under Domain Shift

The experiment retrains only C1:

1) Train C1 on the source domain (Boston scenes). Freeze C2 and C3 permanently.
2) Measure the five-quantity profile on the target-domain validation set (Singapore): C1 detection quality (mAP@50), C2-isolated minADE, C2-pipeline minADE, C3-isolated L2@$\{1,2,3\}$s, C3-pipeline L2@$\{1,2,3\}$s.
3) Fine-tune C1 on the target domain (Singapore scenes), producing the retrained C1$'$.
4) Re-measure the identical five-quantity profile with C1$'$ in place.
5) Verify the isolation property (5) and compute the injected cascades (6) at C2 and C3.

The Boston-to-Singapore shift within nuScenes is a genuine and well-studied domain gap: left- versus right-hand traffic, different vehicle and agent mixes, and different lighting conditions [10].

E. Metrics

C1 is scored with standard detection metrics (mAP@50, mAP@50-95, precision, recall) on the target-domain validation images. For C_2, the metric is the minimum average displacement error (minADE). For an agent with forecast positions $\hat{p}_t^{(m)}$ under mode m and ground-truth positions $p_t^\star$ over T future steps,

$$\text{minADE} = \min_m \frac{1}{T}\sum_{t=1}^{T}\left\|\hat{p}_t^{(m)} - p_t^\star\right\|_2, \tag{9}$$

averaged over agents. Our deterministic predictor has a single mode, so the min collapses to that mode. For C_3, the metric is the L2 displacement error between the planned ego trajectory $\hat{e}_t$ and the human driver's recorded trajectory $e_t^\star$ at horizon τ,

$$\text{L2@}\tau = \left\|\hat{e}_\tau - e_\tau^\star\right\|_2, \qquad \tau \in \{1,2,3\}\ \text{s}, \tag{10}$$

reported together with collision rate. The C3 metric is the standard nuScenes open-loop planning protocol used by the E2E literature [2], [3], [15], with the differential-use caveat discussed in Sections II and VI-C.

IV. Experimental Setup

A. Dataset and Components

We use nuScenes-mini v1.0 [10]. Partitioning by recorded location and splitting each domain into train and validation gives `boston_train`/`boston_val` (2/2 scenes) and `singapore_train`/`singapore_val` (3/3 scenes); all cascade measurements use `singapore_val` ($n = 3$). For C1 training, the 3D box annotations are projected onto the front-camera keyframes to produce 2D labels over eight driving classes.

C1 is YOLOv11n [11], fine-tuned for 10 epochs on `boston_train` from COCO weights, then a further 10 epochs on `singapore_train` starting from the Boston checkpoint; the two checkpoints are the before/after detectors.

At inference, each 2D detection is lifted to an ego-frame ground position by intersecting its bottom-center pixel ray with the ground plane, supplying the agent positions z_1 that C2 consumes. C2 is a deterministic, parameter-free constant-velocity predictor: in isolated mode it rolls agents out at their ground-truth velocity, and in pipeline mode it anchors a zero-velocity rollout at C1's single-frame detections, so that any change in its pipeline output is attributable solely to a change in C1. C3 is a rule-based Intelligent Driver Model [12] proposal-and-score planner that selects, from a speed-aware grid of candidate trajectories $\mathcal{P}$, the one minimizing progress cost plus a soft proximity penalty against the predicted agent futures z_2,

$$\pi^\star = \arg\min_{\pi\in\mathcal{P}}\ J_{\text{prog}}(\pi)\ +\ \lambda\sum_{a\in z_2}\phi\big(d(\pi,a)\big), \tag{11}$$

where $d(\pi,a)$ is the closest approach between candidate π and agent a, ϕ is a soft penalty on proximity, and λ weights safety against progress. This reproduces the PDM-Closed [13] architecture without its learned scorer; the dependence of (11) on z_2 is the channel through which perturbed detections reach the plan.

B. Measurement Protocol

The harness computes the five-quantity profile of Section III-D for both C1 checkpoints on `singapore_val`, with all randomness fixed across the two runs. Because C2 and C3 have no trainable parameters, the isolation property (5) can be checked for exact equality rather than statistical closeness.

V. Results

A. Headline Measurements

Table I reports the complete before/after profile on `singapore_val`, where "before" is the Boston-trained C_1 and "after" is the Singapore-fine-tuned C_1, and Fig. 2 plots it. The detection quality of C_1 (mAP@50) drops after fine-tuning, C2-pipeline minADE improves sharply (15.19 → 3.95 m), and C3-pipeline L2@3s worsens (6.30 → 6.58 m). Both isolated metrics are unchanged, which is the visual signature of the isolation property (5).

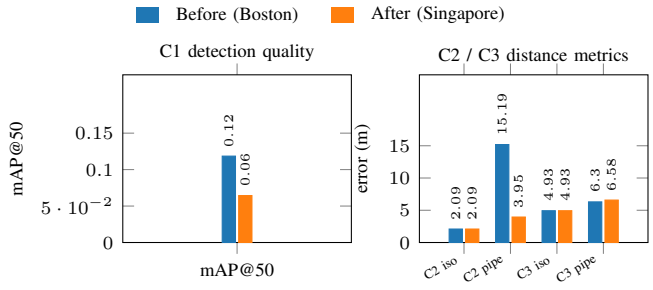


Fig. 2: Full profile on `singapore_val`, before versus after: (a) C1 detection quality, (b) C2/C3 distance metrics.

TABLE I: Measurements on Singapore validation ($n = 3$ scenes).

Metric	Before	After	Δ	Δ%
C1 mAP@50	0.1185	0.0644	-0.0541	-45.66
C1 mAP@50-95	0.0571	0.0258	-0.0313	-54.81
C1 precision	0.6845	0.6709	-0.0136	-1.99
C1 recall	0.0866	0.0477	-0.0389	-44.92
C2 iso. minADE (m)	2.0899	2.0899	0.0000	0.00
C2 pipe. minADE (m)	**15.1930**	**3.9535**	$\mathbf{-11.2395}$	$\mathbf{-73.98}$
C3 iso. L2@1s (m)	0.6900	0.6900	0.0000	0.00
C3 iso. L2@2s (m)	2.4944	2.4944	0.0000	0.00
C3 iso. L2@3s (m)	4.9289	4.9289	0.0000	0.00
C3 pipe. L2@1s (m)	1.0396	1.0838	$+0.0442$	$+4.25$
C3 pipe. L2@2s (m)	3.5017	3.6070	$+0.1053$	$+3.01$
C3 pipe. L2@3s (m)	**6.2969**	**6.5770**	$\mathbf{+0.2801}$	$\mathbf{+4.45}$
C3 pipe. collision rate	0.000	0.000	0.0000	—

B. Verification of the Isolation Property (Finding F2)

The three isolated-mode rows are exactly zero-delta. C2-isolated minADE is 2.0899 m in both runs, and C3-isolated L2 is $0.6900/2.4944/4.9289$ m at the three horizons in both runs, to every recorded digit. For deterministic, parameter-free C2 and C3 fed identical ground-truth inputs, this invariance is expected by construction; we report it as a validity check on the apparatus rather than as a discovery, since any nonzero delta would have signalled state or data leakage between the runs. Its role is to certify that the pipeline-mode deltas in Table I arise solely from the C1 retraining event, transmitted through the component interfaces, the premise on which the injected-cascade readings depend.

C. Cascade Concentration at the C1→C2 Interface (Finding F3)

The largest single effect in Table I occurs at the C1→C2 interface. With the Boston-trained C1, C2's pipeline-mode minADE was 15.1930 m against an isolated-mode floor of 2.0899 m, a cascade gap Γ_2 of 13.10 m. This Γ_2 conflates two sources: C1 detection error and the loss of velocity information, since C2-isolated uses ground-truth velocity whereas C2-pipeline uses a zero-velocity rollout (Section IV); it therefore bounds the combined effect, not perception error alone. After the Singapore fine-tune, C2-pipeline minADE fell to 3.9535 m ($\Gamma_2 = 1.86$ m), a 73.98% improvement without modifying C2. The change $\Delta_2 = -11.24$ m is attributable to C1 alone, since the velocity-information gap is constant across the two runs, so the first downstream interface dominates the retraining-induced cascade.

D. Simultaneous Degradation at the System Output (Finding F4)

Even though C2-pipeline improved by 73.98%, C3-pipeline worsened at every horizon: $+4.25\%$ at 1 s, $+3.01\%$ at 2 s, and $+4.45\%$ at 3 s (6.2969 m $\rightarrow$ 6.5770 m, $\Delta_3 = +0.2801$ m). The cascade gap at the planner grew from $\Gamma_3 = 1.37$ m to 1.65 m, with collision rate zero in both runs. The two pipeline-mode deltas therefore carry opposite signs: the retraining event that improved the planner's inputs degraded its output. Read against the diagnostic (7), the run satisfies the isolation clause ($\Delta_3^{\text{iso}} = 0$) and the degradation clause ($\Delta_3 = +0.28$ m $> \epsilon$) but *not* the upstream-improvement clause, because aggregate mAP fell (F5). The run is therefore not a clean instance of entangled enhancement in the strict sense of the base paper [8], which requires $\delta_1 > 0$; it is the related non-monotone regime, in which C2's interface improves while the system output degrades. It is nonetheless a controlled, quantified instance of the correction-cascade and improvement-deadlock pattern that the technical-debt literature describes only qualitatively [7]. Fig. 3 illustrates the effect: C3-isolated remains fixed at 4.93 m, since the planner receives identical ground-truth inputs in both runs, while C3-pipeline drifts from 6.30 to 6.58 m.

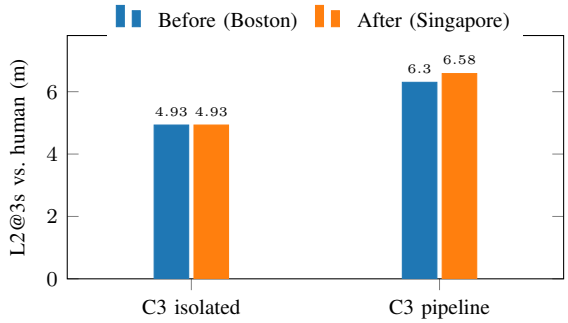


Fig. 3: Cascade signature on planning: isolated fixed, pipeline drifts.

E. Aggregate Metric Decoupling at C1 (Finding F5)

Table I exhibits a second inversion. The Singapore-fine-tuned C1, the checkpoint that improved C2 by 74%, scores *lower* on its own aggregate validation metric than its Boston-trained predecessor (mAP@50 $0.1185 \rightarrow 0.0644$, -45.66%; recall $0.0866 \rightarrow 0.0477$). The immediate cause is small-data overfitting: the Singapore fine-tune set contains only 119 images (944 instances) with a sparse class mix, and the fine-tuned detector converges to detection regimes that diverge from the validation label distribution, as evident in the confusion matrix of Fig. 4, where most non-car classes collapse into the background row. The cascade, however, does not propagate through the aggregate score but through the detection outputs themselves: a detector can lose mAP while emitting outputs that are more useful to its downstream consumer on the relevant scenes. Aggregate component metrics and interface behavior therefore decouple, here moving in opposite directions with large magnitudes on both sides, so a component's own metric is not a reliable admission criterion. This is the per-component, system-scale analogue of the negative-flip phenomenon reported for classifier updates [17].

F. Training Diagnostics

Both fine-tunes converged within their 10-epoch budgets with monotone loss decay, as Fig. 5 shows for the Boston run:

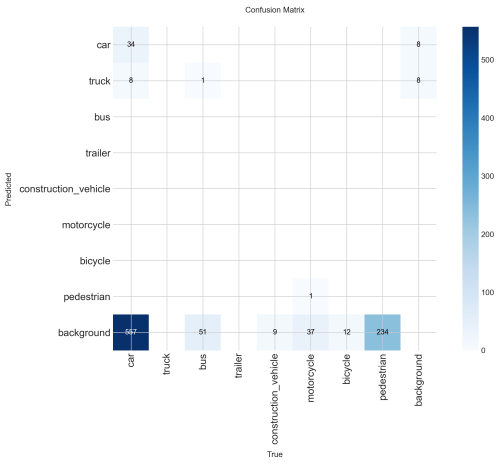


Fig. 4: Confusion matrix of the Singapore-fine-tuned C_1.

the box, classification, and distribution-focal losses decrease on both the train and validation splits, while precision, recall, and mAP@50 increase, with mAP@50 rising from 0.039 at epoch 1 to 0.119 at epoch 10. Each fine-tune completes in under a GPU-minute, several orders of magnitude below the cost of a single E2E training run [6], which is what renders a study of repeated retraining feasible. Fig. 6 shows a qualitative C_1 output batch; the cascade propagates through the ego-frame positions of the detected boxes, which alter the forecasts and hence the plan, independently of whether the detections differ visibly.

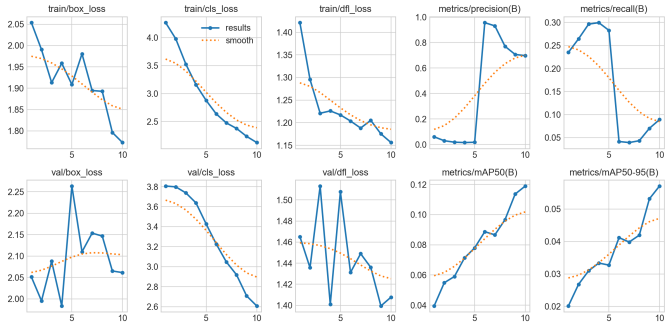
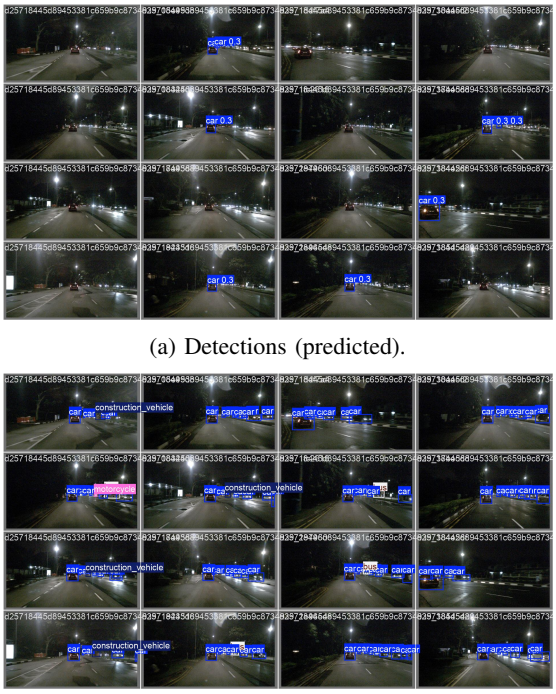


Fig. 5: YOLOv11n C_1 Boston fine-tune training curves.

VI. Discussion

A. Mechanism of the Non-Monotone Cascade

Under the Boston-trained C1, the planner's inputs were substantially displaced ($\Gamma_2 = 13.10$ m) in a regime whose errors the proposal-and-score rules absorbed: missed or grossly misplaced agents leave the proximity penalty inactive, so the planner follows its kinematic prior, which on these scenes approximated the human trajectory. The fine-tuned C1 relocated detections near their true positions and re-activated the penalty with correctly anchored but coarse (zero-velocity) forecasts, causing the planner to react to obstacles it had previously ignored. Downstream components can thus absorb



(a) Detections (predicted).

(b) Ground-truth labels.

Fig. 6: Qualitative C_1 output on a Singapore validation batch.

upstream error, and, symmetrically, upstream improvement can expose masked downstream sensitivity, so a retrained component cannot be admitted on its own interface alone.

This mechanism suggests the C3 regression may be a metric artifact rather than a loss of driving quality. Collision rate was zero before and after (Table I), and the worse L2 arises because the improved detector made the planner react to real agents instead of following a kinematic prior that happened to track the recorded human path. This is the ego-forecasting shortcut of nuScenes open-loop L2, under which perception-agnostic prediction is competitive [15] and open-loop error is weakly tied to closed-loop quality [14]. The update may thus be a *safer* planner that scores worse on a metric rewarding ignored perception. This directly affects CARA (Section VII): since its admission rule keys on the terminal open-loop metric, it could reject a genuinely improved update, so a closed-loop or NAVSIM-style metric [16] is the appropriate safeguard. Until then the $+4.45\%$ should be read as reduced trajectory agreement, not an established safety regression.

B. Implications for MLS Maintenance Models

The isolation check (F2) validates the per-component state factorization that stochastic maintenance models assume. The injected cascades $\Delta_2 = -11.24$ m and $\Delta_3 = +0.28$ m have opposite signs, demonstrating that a single scalar "retraining quality" parameter is insufficient: one event can act simultaneously as a repair for C2 and a fault injection for C3. Models of imperfect retraining [9] can represent this through state-dependent transitions, which the coupling factor

of Section VII is intended to calibrate once measured over updates with an unambiguous δ_i.

C. Threats to Validity

Statistical power. The validation set contains 3 scenes, one of which dominates the C3 regression. The isolation rows establish that the $+4.45\%$ figure is a genuine measurement rather than apparatus noise, but its generalization is not established; multi-seed replication and larger splits are the direct remedy.

Construct validity of open-loop L2. As discussed in Section VI, open-loop L2 rewards trajectory agreement over safety, so the C3 result must be read differentially and confirmed with a closed-loop or NAVSIM-style metric [16] before any safety claim.

Component realism. C2 is parameter-free and C1 is camera-based rather than LiDAR [21], and the IDM planner omits PDM-Closed's learned scorer [13]; magnitudes will differ with richer components, though the existence results (F2–F5) are architecture-independent.

Small-data fine-tuning. The mAP drop in F5 arises from a 119-image fine-tune; a class-balanced fine-tune could flip the sign of δ_1, which would reinforce F5 (the decoupling claim) and, moreover, yield the unambiguous δ_1 that a clean ρ requires.

VII. Proposed Method: Cascade-Aware Retraining Assessment

The findings expose a concrete hazard: a retrained component that regresses on its own metric may nonetheless improve its neighbor while degrading the system (F4, F5), so neither component-level validation nor first-interface validation is a sufficient admission criterion. We therefore generalize the measurement method of this paper into an admission protocol for component updates in modular MLS, termed Cascade-Aware Retraining Assessment (CARA), which formalizes the procedure carried out manually in our experiment. Fig. 7 summarizes the decision; the protocol is stated below.

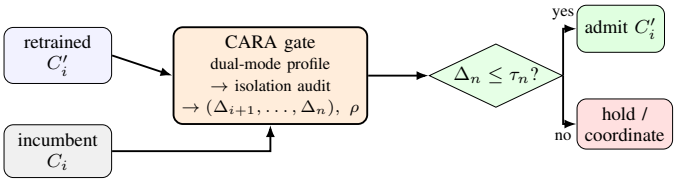


Fig. 7: The CARA admission gate for a retrained component.

A. Protocol

Given a pipeline $C_1 \to \cdots \to C_n$, a frozen audit set A, and a candidate update R to component C_i, CARA proceeds as follows (Algorithm 1). With the incumbent C_i it records the dual-mode profile $P_{\text{before}} = \{M_k^{\text{iso}}, M_k^{\text{pipe}} : k = i..n\}$ on A, then swaps in the retrained C_i' and records P_{after} on the identical audit set with fixed randomness. It requires $M_k^{\text{iso}}(\text{before}) = M_k^{\text{iso}}(\text{after})$ for every frozen C_k; any violation signals leakage or nondeterminism and invalidates the assessment (in our experiment this audit passed bit-exactly, F2). It then computes the injected cascade Δ_k (6) at every downstream component and reports the full vector $(\Delta_{i+1}, \ldots, \Delta_n)$, since adjacent deltas can carry opposite signs, together with the cascade coupling factor of Section III-C, generalized to an update on any component,

$$\rho_{i \to n} = \frac{\Delta_n}{\delta_i}. \tag{12}$$

A negative ρ with an improving δ_i is the entangled-enhancement regime. We stress that ρ is well-defined only when δ_i is unambiguous; in our single experiment $\delta_1 < 0$, so $\rho_{1 \to 3}$ is of ambiguous sign and is not cleanly measurable, and a meaningful value requires an update whose component metric moves definitely, such as the class-balanced C1 fine-tune of Section VIII. Finally, C_i' is admitted only if $\Delta_n \le \tau_n$ and safety-critical auxiliaries such as collision rate do not regress; if $\Delta_n > \tau_n$ while intermediate interfaces improve, as in the observed pattern, the update is held or routed to coordinated retraining of the downstream consumers, converting a single-component update into a coordinated multi-component transition.

Algorithm 1: Cascade-Aware Retraining Assessment (CARA)

Input: pipeline $C_1 \to \ldots \to C_n$; audit set A; candidate update R on C_i; tolerance τ_n
Output: admit / hold decision, cascade vector, coupling factor ρ

$P_{\text{before}} \leftarrow \textsc{DualModeProfile}(C_i, A)$
$C_i' \leftarrow R(C_i)$; $P_{\text{after}} \leftarrow \textsc{DualModeProfile}(C_i', A)$
foreach *frozen* C_k, $k > i$ **do**
  if $M_k^{iso}(\textit{after}) \neq M_k^{iso}(\textit{before})$ **then**
    return invalid `// isolation violated (contamination)`
  end
end
$\Delta_k \leftarrow M_k^{\text{pipe}}(\text{after}) - M_k^{\text{pipe}}(\text{before})$ for $k = i{+}1..n$
$\rho \leftarrow \Delta_n / \delta_i$
if $\Delta_n \le \tau_n$ ***and*** *collision rate not worse* **then**
  return admit C_i'
else
  return hold / coordinate downstream retraining
end

B. Mitigation of the Observed Failure Modes

Admission never rests on δ_i (addressing F5) and reports the full delta vector, so an improvement at C2 cannot mask a regression at C3 (addressing F4). The isolation audit in Step 3 makes silent contamination detectable by exact equality testing, a check available only because the architecture is modular (F1), and CARA's marginal cost is inference on a fixed audit set with no training. In an E2E monolith Steps

1–5 are undefined, since there are no frozen components, isolated modes, or interfaces at which Δ_k could be measured, so the protocol is constructible only in the modular setting.

C. Positioning

CARA is largely the experimental procedure of this paper cast as an explicit gate; its contribution is not a new algorithm but the gate statistic, a per-interface cascade vector with a terminal admission threshold and an isolation audit, in place of input-data drift scores [19] alone. It is complementary to regression-constrained retraining [17], [18], which *reduces* the cascade that CARA *measures*, and to uncertainty-propagating interfaces [20] that absorb the residual.

VIII. Conclusion and Future Work

We presented a controlled study of single-component retraining in a modular perception–prediction–planning pipeline under a geographic domain shift. Under dual-mode evaluation with a verified isolation control, retraining C1 improved C2 by 73.98% in pipeline mode while C3 degraded by 4.45% and C1's aggregate metric moved opposite to its interface usefulness. These findings, from the architectural necessity of modularity to aggregate-metric decoupling, show that component-level validation is insufficient for safe MLS maintenance, which the CARA protocol addresses at inference-only cost. The broader contribution is methodological: cascade effects can be measured under exact controls, but only in an architecture that exposes its interfaces.

Future work runs along four lines: statistical hardening with multi-seed replication and larger splits; class-balanced fine-tuning of C1, which both tests the entangled-enhancement signature and yields the unambiguous δ_1 a clean ρ requires; full coverage of the seven-subset retraining lattice over $\{$C1, C2, C3$\}$, calibrating the base paper's stochastic maintenance models [8], [9]; and moving C1 to LiDAR detection [21] with NAVSIM-style scoring [16] for C3.

References

[1] L. Chen, P. Wu, K. Chitta, B. Jaeger, A. Geiger, and H. Li, "End-to-end autonomous driving: Challenges and frontiers," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.

[2] Y. Hu, J. Yang, L. Chen, K. Li, C. Sima, X. Zhu, S. Chai, S. Du, T. Lin, W. Wang, L. Lu, X. Jia, Q. Liu, J. Dai, Y. Qiao, and H. Li, "Planning-oriented autonomous driving," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[3] B. Jiang, S. Chen, Q. Xu, B. Liao, J. Chen, H. Zhou, Q. Zhang, W. Liu, C. Huang, and X. Wang, "Vad: Vectorized scene representation for efficient autonomous driving," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[4] E. Ma, L. Zhou, T. Tang, J. Zhang, J. Jiang, Z. Zhang, D. Han, K. Zhan, X. Zhang, X. Lang, H. Sun, X. Zhou, D. Lin, and K. Yu, "Correctad: A self-correcting agentic system to improve end-to-end planning in autonomous driving," *arXiv preprint arXiv:2511.13297*, 2025.

[5] H. Liu, T. Li, H. Yang, L. Chen, C. Wang, K. Guo, H. Tian, H. Li, H. Li, and C. Lv, "Reinforced refinement with self-aware expansion for end-to-end autonomous driving," *arXiv preprint arXiv:2506.09800*, 2025.

[6] W. Sun, X. Lin, Y. Shi, C. Zhang, H. Wu, and S. Zheng, "Sparsedrive: End-to-end autonomous driving via sparse scene representation," *arXiv preprint arXiv:2405.19620*, 2024.

[7] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 28, 2015, pp. 2503–2511.

[8] Z. Wang and F. Machida, "Maintaining performance of a machine learning system against imperfect retraining," in *2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 2024, pp. 1782–1787.

[9] Z. Wang and F. Machida, "Exploiting the availability-continuity trade-off in imperfect retraining of machine learning systems," in *2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*, 2025, pp. 406–417.

[10] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, "nuscenes: A multimodal dataset for autonomous driving," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 11 621–11 631.

[11] G. Jocher and J. Qiu, "Ultralytics yolo11," https://github.com/ultralytics/ultralytics, 2024.

[12] M. Treiber, A. Hennecke, and D. Helbing, "Congested traffic states in empirical observations and microscopic simulations," *Physical Review E*, vol. 62, no. 2, pp. 1805–1824, 2000.

[13] D. Dauner, M. Hallgarten, A. Geiger, and K. Chitta, "Parting with misconceptions about learning-based vehicle motion planning," in *Conference on Robot Learning (CoRL)*, 2023.

[14] F. Codevilla, A. M. López, V. Koltun, and A. Dosovitskiy, "On offline evaluation of vision-based driving models," in *European Conference on Computer Vision (ECCV)*, 2018.

[15] J.-T. Zhai, Z. Feng, J. Du, Y. Mao, J.-J. Liu, Z. Tan, Y. Zhang, X. Ye, and J. Wang, "Rethinking the open-loop evaluation of end-to-end autonomous driving in nuscenes," *arXiv preprint arXiv:2305.10430*, 2023.

[16] D. Dauner, M. Hallgarten, T. Li, X. Weng, Z. Huang, Z. Yang, H. Li, I. Gilitschenski, B. Ivanovic, M. Pavone, A. Geiger, and K. Chitta, "Navsim: Data-driven non-reactive autonomous vehicle simulation and benchmarking," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[17] S. Yan, Y. Xiong, K. Kundu, S. Yang, S. Deng, M. Wang, W. Xia, and S. Soatto, "Positive-congruent training: Towards regression-free model updates," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 14 299–14 308.

[18] Y. Shen, Y. Xiong, W. Xia, and S. Soatto, "Towards backward-compatible representation learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 6368–6377.

[19] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data validation for machine learning," in *Proceedings of Machine Learning and Systems (SysML/MLSys)*, vol. 1, 2019.

[20] B. Ivanovic, Y. Lin, S. Shrivastava, P. Chakravarty, and M. Pavone, "Propagating state uncertainty through trajectory forecasting," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2022.

[21] T. Yin, X. Zhou, and P. Krähenbühl, "Center-based 3d object detection and tracking," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 11 784–11 793.